

Ejercicio Spring Boot — Sistema de Donación de Sangre

Objetivo

Desarrollar una API REST profesional utilizando **Java 21 + Spring Boot 3** para gestionar el proceso de donación de sangre en un banco de sangre.

La aplicación debe aplicar:

- Principios SOLID
- Clean Code
- Arquitectura por capas
- DTO Pattern
- Validaciones
- Manejo global de excepciones
- Persistencia con MySQL
- Firma de consentimiento del donante

Contexto del Problema

Un banco de sangre necesita digitalizar el proceso de registro y control de donaciones.

El sistema debe permitir:

- Registrar donantes
- Gestionar jornadas de donación
- Registrar donaciones
- Validar requisitos médicos
- Registrar firma de consentimiento
- Consultar historial de donaciones
- Controlar disponibilidad de sangre

Requerimientos Funcionales

1. Gestión de Donantes

Registrar donante

POST /api/donantes

Campos:

- nombres
- apellidos
- documento
- fechaNacimiento
- tipoSangre
- peso
- telefono
- correo
- direccion
- fechaUltimaDonacion
- aceptaConsentimiento
- firmaConsentimiento

2. Firma de Consentimiento

El donante debe aceptar el consentimiento informado antes de donar sangre.

Requisitos

- El campo aceptaConsentimiento debe ser obligatorio.
- La firma puede almacenarse como:
 - Base64
 - URL del archivo

- Ruta del documento PDF
- No se puede registrar una donación sin consentimiento firmado.

3. Gestión de Donaciones

Registrar donación

POST /api/donaciones

Campos:

- donanteld
- cantidadML
- fechaDonacion
- observaciones

Reglas de Negocio

Validaciones médicas

No permitir donar si:

- El donante es menor de 18 años
- Pesa menos de 50 kg
- Han pasado menos de 3 meses desde la última donación
- No aceptó consentimiento
- No tiene firma registrada

Reglas adicionales

- Registrar automáticamente la fecha de donación
- Actualizar fechaUltimaDonacion
- Generar código único de donación

- Registrar historial de donaciones

Gestión de Inventario

Tipos de sangre

- O+
- O-
- A+
- A-
- B+
- B-
- AB+
- AB-

Funcionalidades

Consultar inventario

GET /api/inventario

Consultar por tipo de sangre

GET /api/inventario/{tipo}

Arquitectura Obligatoria

El proyecto debe seguir arquitectura limpia:

controller

service

service.impl

repository

domain

dto

mapper
exception
config
util
enums

Principios SOLID

S — Single Responsibility

Cada clase debe tener una única responsabilidad.

Ejemplo:

- DonanteService → lógica de negocio
- DonanteRepository → acceso a datos
- ConsentimientoService → manejo de firmas

O — Open/Closed

Permitir agregar nuevas validaciones médicas sin modificar lógica existente.

Ejemplo:

```
public interface ValidacionDonante {  
    void validar(Donante donante);  
}
```

L — Liskov Substitution

Las implementaciones deben sustituir interfaces sin afectar el comportamiento.

I — Interface Segregation

Interfaces pequeñas y enfocadas.

D — Dependency Inversion

Usar inyección de dependencias mediante interfaces.

```
private final DonacionService donacionService;
```

Requisitos de Clean Code

- Métodos pequeños y legibles
- Nombres descriptivos
- Evitar lógica en controllers
- Código modular
- Evitar duplicación
- Responsabilidad clara por clase

DTOs

Requests

- DonanteRequest
- DonacionRequest
- ConsentimientoRequest

Responses

- DonanteResponse
- DonacionResponse
- InventarioResponse

Nunca retornar entidades directamente.

Validaciones

Usar Bean Validation:

@NotBlank

@NotNull

@Positive

@Email

@Past

Ejemplos:

- Documento obligatorio
- Correo válido
- Peso positivo
- Fecha válida

Excepciones Personalizadas

Crear:

- ResourceNotFoundException
- BusinessException
- ConsentimientoNoFirmadoException
- DonanteNoAptoException

Implementar:

@RestControllerAdvice

Base de Datos MySQL

Nombre:

blood_bank_db

Tablas sugeridas:

- donors
- donations
- consents

- blood_inventory

Tecnologías Obligatorias

- Java 21
- Spring Boot 3
- Spring Web
- Spring Data JPA
- MySQL
- Lombok
- MapStruct
- Validation
- Swagger/OpenAPI
- SLF4J

Funcionalidades Extra

Nivel Intermedio

- Paginación
- Filtros por tipo de sangre
- Exportar historial PDF

Nivel Avanzado

- JWT Authentication
- Subida de firma como imagen
- Docker
- Tests unitarios con Mockito

- Tests de integración
- Auditoría de cambios

Endpoints Sugeridos

Donantes

POST /api/donantes
GET /api/donantes
GET /api/donantes/{id}
PUT /api/donantes/{id}
DELETE /api/donantes/{id}

Donaciones

POST /api/donaciones
GET /api/donaciones
GET /api/donaciones/{id}

Consentimientos

POST /api/consentimientos
GET /api/consentimientos/{donantId}

Inventario

GET /api/inventario
GET /api/inventario/{tipoSangre}